作業系統HW3

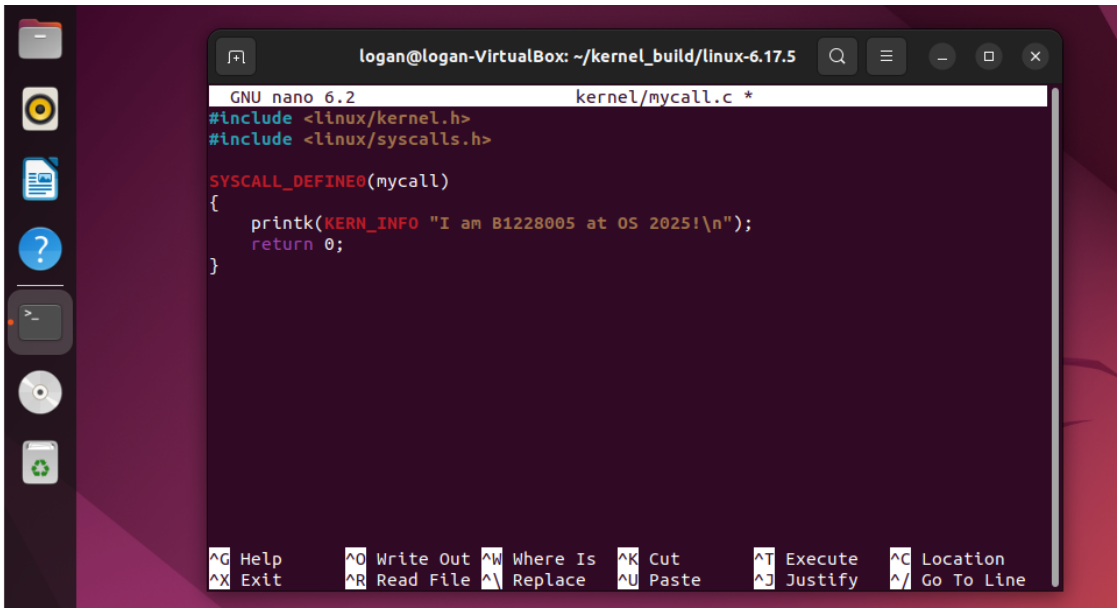
AI三 B1228005 胡樂麒

1. 進入作業二下載過kernel的資料夾

```
logan@logan-VirtualBox:~$ cd ~/kernel_build/linux-6.17.5
```

2. 建立 system call 程式碼(mycall.c)

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo nano kernel/mycall.c
```



3. 新增systemcall 470

```
[sudo] password for logan:
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo nano arch/x86/entry/sys
calls/syscall_64.tbl
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$
```

```
466     common  removexattrat           sys_removexattrat
467     common  open_tree_attr          sys_open_tree_attr
468     common  file_getattr            sys_file_getattr
469     common  file_setattr            sys_file_setattr
470     common  mycall                  sys_mycall
#
```

4. 加入 function prototype

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo nano include/linux/sysc
alls.h
```

```
                                        u32 __user *size, u32 flags);
asmlinkage long sys_lsm_set_self_attr(unsigned int attr, struct lsm_ctx __user >
                                        u32 size, u32 flags);
asmlinkage long sys_lsm_list_modules(u64 __user *ids, u32 __user *size, u32 fla>

asmlinkage long sys_mycall(void);

/*
 * Architecture-specific system calls
 */

/* x86 */
asmlinkage long sys_ioperm(unsigned long from, unsigned long num, int on);

asmlinkage long sys_uretprobe(void);
```

5. Makefile 將 mycall.o 連結到 kernel 。(kernel 編譯時會把mycall.c自動轉換成mycall.o)

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo nano kernel/Makefile
[sudo] password for logan:
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$
```

```
logan@logan-VirtualBox: ~/kernel_build/linux-6.17.5
  GNU nano 6.2                    kernel/Makefile
KMSAN_SANITIZE_kcov.o := n
CFLAGS_kcov.o := $(call cc-option, -fno-conserve-stack) -fno-stack-protector

obj-y += sched/
obj-y += locking/
obj-y += power/
obj-y += printk/
obj-y += irq/
obj-y += rcu/
obj-y += livepatch/
obj-y += dma/
obj-y += entry/
obj-y += unwind/
obj-y += mycall.o
obj-$(CONFIG_MODULES) += module/


obj-$(CONFIG_KCMP) += kcmp.o
obj-$(CONFIG_FREEZER) += freezer.o
obj-$(CONFIG_PROFILING) += profile.o

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

6. 編譯 Kernel, 安裝模組, 安裝kernel本體

使用oldconfig沿用原始設定

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ make oldconfig
#
# No change to .config
#
```

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ make -j$(nproc)
  SYNC    include/config/auto.conf.cmd
  SYSHDR  arch/x86/include/generated/uapi/asm/unistd_64.h
  UPD     include/config/kernel.release
  SYSHDR  arch/x86/include/generated/uapi/asm/unistd_x32.h
```

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo make modules_install
  SYMLINK /lib/modules/6.17.5-myhw3/build
  INSTALL /lib/modules/6.17.5-myhw3/modules.order
```

```
logan@logan-VirtualBox:~/kernel_build/linux-6.17.5$ sudo make install
  INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.17.5-myhw3 /boot/v
mlinuz-6.17.5-myhw3
```

```
logan@logan-VirtualBox:~$ uname -r
6.17.5-myhw3
```

7. 撰寫test_mycall.c來測試 system call

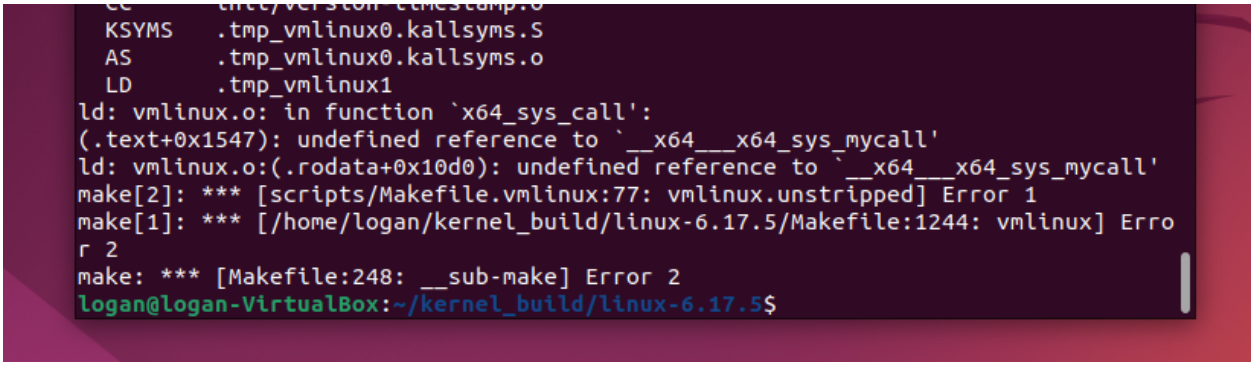
```
logan@logan-VirtualBox:~$ gcc test_mycall.c -o test_mycall
```

```
logan@logan-VirtualBox: ~
  GNU nano 6.2                    test_mycall.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define MYCALL_NUM 470

int main() {
    long ret = syscall(MYCALL_NUM);
    printf("mycall returned: %ld\n", ret);
    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

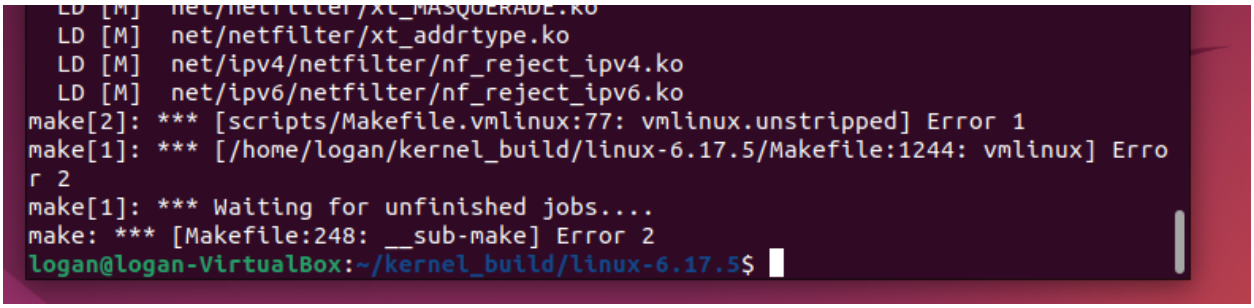
```
logan@logan-VirtualBox:~$ ./test_mycall
mycall returned: 0
logan@logan-VirtualBox:~$ sudo dmesg
[   99.262102] I am B1228005 at OS 2025!
```

8. 遇到的問題

1. system call格式錯誤，我原本這樣打 470 common mycall __x64_sys_mycall，之後改成 470 common mycall sys_mycall，我問ChatGPT為甚麼這樣會出現問題，他說因為我使用的版本比較舊，新版的才需要加__x64的前綴。



2. Makefile 中 obj-y += mycall.o 我一開始以為是擺在上面的區塊，結果編譯kernel時跳error，看下面的參考資料後，我回去看往下滑發現這段程式要擺在下面。



9. 參考資料:

ChatGPT,
手把手編譯linux與新增syscall
新增自定義system call